

Experiment/Practical 10 Ensemble Methods

Title: Implementation of Ensemble Methods for Classification

Aim: To implement and understand the concept of ensemble methods for classification tasks and evaluate their performance on a given dataset.

Objective: Students will learn

- The basic concept of ensemble learning and how it improves classification performance.
 - Various ensemble methods, including bagging, boosting, and stacking.
 - Implementing popular ensemble algorithms such as Random Forest, AdaBoost, and Gradient Boosting.
 - Evaluating the model's performance using relevant metrics such as accuracy, precision, recall, and F1-score.
 - Comparing the performance of ensemble models with individual classifiers.
-

Problem statement

Use the given dataset(s) to demonstrate the application of ensemble methods for classification tasks. The task is to improve the classification performance by combining multiple weak classifiers to create a strong classifier using ensemble techniques.

Explanation/Stepwise Procedure/ Algorithm:

- Give a brief description of the ensemble techniques /methods

Ensemble methods combine several trees base algorithms to construct better predictive performance than a single tree base algorithm. The main principle behind the ensemble model is that a group of weak learners come together to form a strong learner, thus increasing the accuracy of the model. When we try to predict the target variable using any machine learning technique, the main causes of difference in actual and predicted values are noise, variance, and bias. Ensemble helps to reduce these factors (except noise, which is irreducible error).

- Describe types of ensemble techniques (Bagging, Boosting, Stacking)

Bagging, also known as bootstrap aggregating

Is the process of training several models concurrently on a random subset of the data (with replacement), then averaging the predictions from each model. Another well-known example of bagging is Random Forest.

- Give mathematical formulation (Bagging, Boosting, Stacking)
 - A. **Bagging** (Bootstrap Aggregating): Trains multiple models (e.g., decision trees) independently on random samples of the data (with replacement). Reduces variance and overfitting.
 - Popular Algorithm: Random Forest.
 - Random Forest: Uses an ensemble of decision trees trained on different data and features. Final prediction is majority vote (classification) or average (regression).
 - B. **Boosting**: Builds models sequentially. Each new model focuses on correcting the errors made by previous ones. Weights are adjusted to focus on harder-to-classify instances. Reduces bias and variance.
 - Popular Algorithms: AdaBoost, Gradient Boosting, XGBoost, LightGBM.
 - AdaBoost: Increases the weight of misclassified samples and combines weak learners into a strong classifier.
 - Gradient Boosting: Builds new models to predict the residual errors of the previous models.
 - C. **Stacking**: Trains multiple diverse models (base learners), then combines their outputs using a meta-learner (e.g., Logistic Regression). Can outperform other ensemble methods if base learners are diverse.
 - Base-level models could be Random Forest, SVM, and KNN. The meta-model learns how to best combine their predictions.
- Write the importance of the ensemble techniques in the classification task
- Improve accuracy and generalization.
- Less prone to overfitting (especially bagging).
- Boosting is great for imbalanced data or hard-to-classify points.
- Useful when single models fail to achieve good accuracy.
- Mention applications of ensemble techniques
- **Healthcare**: Disease prediction using patient data.
- **Finance**: Credit scoring and fraud detection.
- **E-commerce**: Product recommendation engines.
- **Cybersecurity**: Intrusion detection systems.
- **Image/Voice Recognition**: Ensembling CNNs for higher accuracy.
- Brief explanation of performance metrics (e.g., accuracy, precision, recall, f1-score, confusion matrix, cross-validation)

Accuracy

Accuracy is the proportion of all classifications that were correct, whether positive or negative. It is mathematically defined as:

$$\text{Accuracy} = \frac{\text{correct classifications}}{\text{total classifications}} = \frac{TP + TN}{TP + TN + FP + FN}$$

In the spam classification example, accuracy measures the fraction of all emails correctly classified.

Recall, or true positive rate

The true positive rate (TPR), or the proportion of all actual positives that were classified correctly as positives, is also known as recall.

Recall is mathematically defined as:

$$\text{Recall (or TPR)} = \frac{\text{correctly classified actual positives}}{\text{all actual positives}} = \frac{TP}{TP + FN}$$

False positive rate

The false positive rate (FPR) is the proportion of all actual negatives that were classified incorrectly as positives, also known as the probability of false alarm. It is mathematically defined as:

$$\text{FPR} = \frac{\text{incorrectly classified actual negatives}}{\text{all actual negatives}} = \frac{FP}{FP + TN}$$

Precision

Precision is the proportion of all the model's positive classifications that are actually positive. It is mathematically defined as:

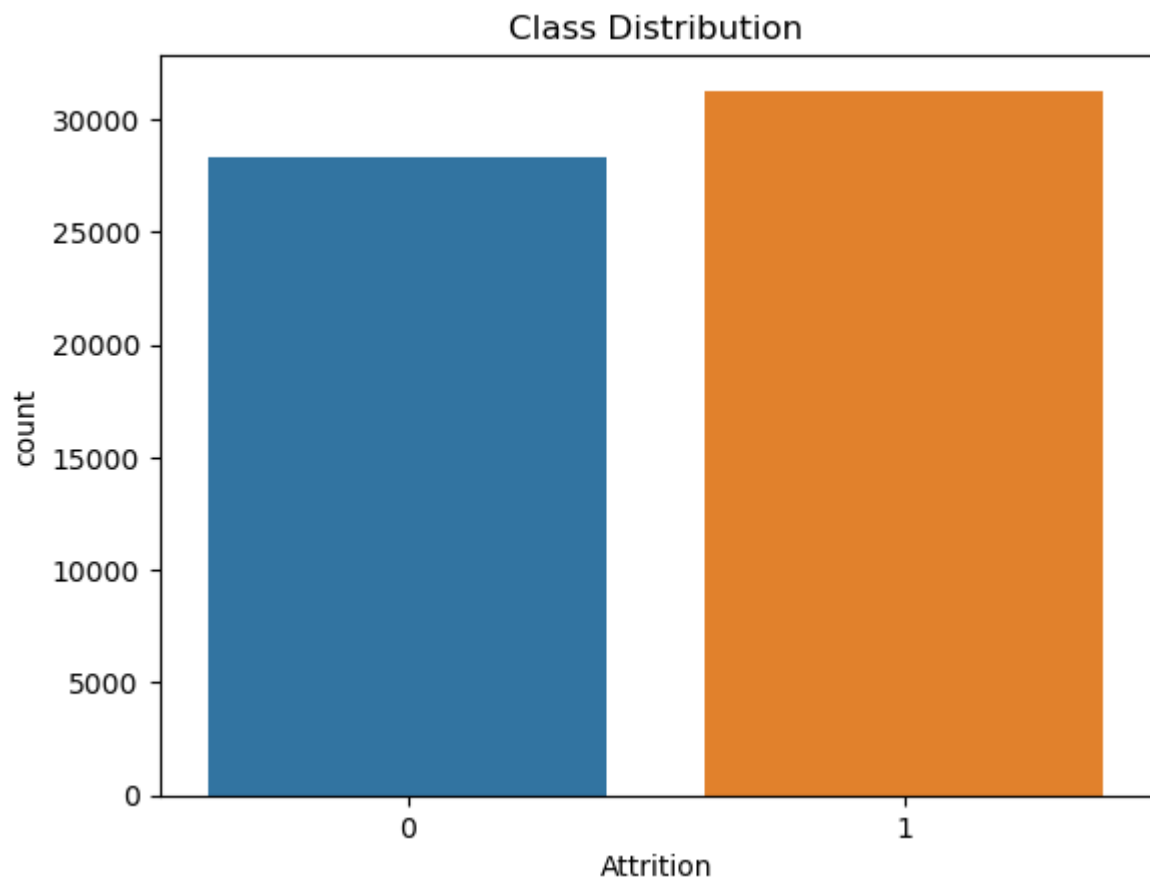
$$\text{Precision} = \frac{\text{correctly classified actual positives}}{\text{everything classified as positive}} = \frac{TP}{TP + FP}$$

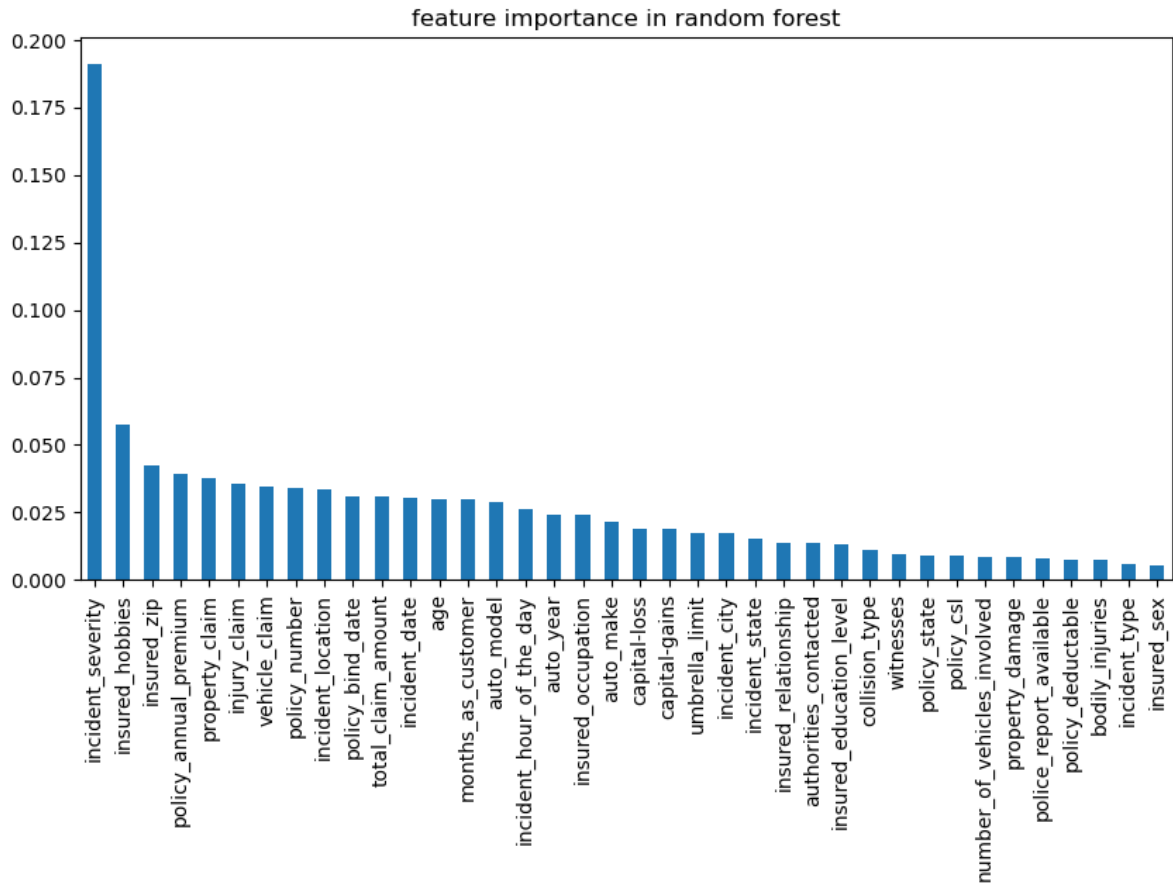
- *Add necessary figure(s)/Diagram(s)*

Accuracy on Test Set: 0.74

	precision	recall	f1-score	support
0	0.73	0.73	0.73	5667
1	0.75	0.76	0.76	6253
accuracy			0.74	11920
macro avg	0.74	0.74	0.74	11920
weighted avg	0.74	0.74	0.74	11920

[[4110 1557]
[1514 4739]]





Accuracy on Test Set: 0.77

	precision	recall	f1-score	support
0	0.80	0.90	0.85	145
1	0.61	0.40	0.48	55
accuracy			0.77	200
macro avg	0.70	0.65	0.67	200
weighted avg	0.75	0.77	0.75	200

```
[[131 14]
 [ 33 22]]
```

```

Accuracy on Test Set: 0.77
      precision    recall  f1-score   support

     0       0.80      0.90      0.85      145
     1       0.61      0.40      0.48       55

 accuracy
macro avg       0.70      0.65      0.67      200
weighted avg     0.75      0.77      0.75      200

[[131  14]
 [ 33  22]]

```

- **Conclusion:**

Impact of Hyperparameters

1. Number of Base Models:

In ensemble methods like Random Forest and Boosting, increasing the number of base learners (trees) generally improves accuracy and reduces variance. However, too many models can increase computation time without significant gains and may risk overfitting in some boosting scenarios.

2. Learning Rate (Boosting):

The learning rate controls how much each tree contributes to the final prediction. A lower learning rate tends to improve generalization and model robustness, though it requires more trees (iterations) to converge. A higher learning rate speeds up training but can lead to overfitting.

3. Tree Depth (Random Forest):

Deeper trees can model complex patterns but are more prone to overfitting. Shallower trees are faster and more interpretable but may underfit. Setting an optimal `max_depth` ensures a balance between bias and variance.

Feature	Random Forest	Boosting (e.g., AdaBoost, XGBoost)
Ensemble Type	Bagging	Boosting
Base Learner	Decision Trees	Shallow Trees (often depth=1)
Training Strategy	Parallel	Sequential (each corrects previous)
Bias-Variance Trade-	Reduces variance	Reduces bias

Feature	Random Forest	Boosting (e.g., AdaBoost, XGBoost)
off		
Overfitting Tendency	Less prone (with enough trees)	Can overfit if not tuned properly
Interpretability	Medium	Low (especially for large number of trees)
Speed	Faster to train (can parallelize)	Slower (sequential dependency)
Accuracy	High	Often higher (especially on structured data)